RELATÓRIO – SISTEMA INTELIGENTE DE GERENCIAMENTO DE ENERGIA DA COLÔNIA

Nome: Stela Silva Santos **RM:** 574593
Fase 3 - Cap 1 - A Aurora Estabelece os Primeiros Sistemas da Colônia

1. Introdução e objetivo

O projeto apresenta um sistema integrado, desenvolvido em Python, para simular o gerenciamento inteligente de energia de uma colônia. O programa organiza informações sobre geração, consumo, reserva de energia, clima e subsistemas. A partir desses dados, aplica regras para emitir alertas e recomendações e utiliza regressão linear simples para estimar a geração eólica.

O objetivo é demonstrar como dados organizados podem ser transformados em decisões claras. Os valores utilizados são fictícios e servem para fins didáticos.

2. Organização dos dados

Foram utilizadas estruturas básicas da linguagem Python, sem bibliotecas externas:

Dicionários e estrutura hierárquica

O dicionário SISTEMAS_COLONIA organiza os sistemas em níveis. No nível principal estão o sistema energético, o suporte à vida e os sistemas não essenciais. Dentro do sistema energético estão os subsistemas solar, eólico e bateria. Essa organização facilita localizar e consultar informações de cada parte da colônia.

Outro dicionário, DADOS_ATUAIS, guarda as leituras atuais em pares chave-valor, por exemplo: energia disponível kwh = 40, geracao_kwh = 40 e consumo_kwh = 70.

Listas de dados históricos

O histórico da previsão é armazenado em duas listas relacionadas pela posição: vento_ms = [8, 10, 12] e geracao_kwh = [20, 25, 30]. Cada velocidade do vento corresponde à geração registrada na mesma posição da outra lista.

3. Regras de decisão utilizadas

As regras ficam no arquivo decisoes.py. Primeiro, o programa registra que os serviços essenciais – oxigênio, água e controle de temperatura – devem ser mantidos ligados. Em seguida, avalia os níveis de energia e consumo:

- **Energia abaixo de 50 kWh e consumo acima de 60 kWh:** emite alerta crítico e recomenda ativar o modo economia.
- **Energia abaixo de 50 kWh**: recomenda reduzir o consumo não essencial.
- **Consumo acima de 60 kWh:** recomenda verificar equipamentos e reduzir cargas não essenciais.
- **Demais situações:** informa que os níveis estão dentro dos limites didáticos definidos.

A prioridade é preservar o suporte à vida. Por isso, a economia deve começar por sistemas não essenciais, como entretenimento e iluminação decorativa, e não pelos serviços indispensáveis.

4. Análise do balanço energético

O programa compara a geração com o consumo usando a operação saldo = geração – consumo. Se o saldo for negativo, existe déficit e o sistema verifica se a reserva informada pode cobri-lo. Se for positivo, existe excedente e o programa sugere armazená-lo, caso haja capacidade na bateria. Se for zero, geração e consumo estão equilibrados.

Exemplo aplicado

Com geração de 40 kWh e consumo de 70 kWh, o saldo é 40 – 70 = –30 kWh. Portanto, há um déficit de 30 kWh. Como o exemplo considera reserva de 30 kWh, o sistema informa que a reserva pode cobrir o déficit atual. Isso é uma análise simplificada de um período; na operação real também seria necessário considerar perdas, limites de potência e capacidade disponível da bateria.

5. Modelo de previsão:

regressão linear simples A previsão de geração eólica utiliza uma reta no formato y = a·x + b. A variável x representa a velocidade do vento em metros por segundo (m/s), e y representa a geração estimada em kWh. O programa calcula os coeficientes a (inclinação) e b (intercepto) a partir dos dados históricos, usando o método dos mínimos quadrados.

Com os pares (8, 20), (10, 25) e (12, 30), a relação encontrada é y = 2,5x. Assim, para uma velocidade de vento de 11 m/s: y = 2,5 × 11 = 27,5 kWh. A saída esperada é uma previsão aproximada de 27,5 kWh.

A regressão é simples e adequada ao objetivo didático. Ela não considera fatores como eficiência da turbina, direção do vento, turbulência, manutenção ou limites do equipamento. Para melhorar a previsão, seria necessário reunir mais medições reais.

6. Funcionamento integrado do programa

O arquivo main.py coordena as etapas: importa os dados, apresenta a hierarquia dos sistemas, mostra as leituras, executa as regras de decisão, calcula o balanço energético e chama a função de previsão. A separação em arquivos e funções deixa o código mais organizado e facilita alterações.

Exemplo de entrada

```
Energia disponível: 40 kWh
Geração: 40 kWh
Consumo: 70 kWh
Vento: 11 m/s
Reserva da bateria: 30 kWh
```

Exemplo de saída

```
PRIORIDADE: manter oxigênio, água e controle de temperatura
ligados.
ALERTA CRÍTICO: energia baixa e consumo alto. Ativar modo
economia.
AÇÃO: reduzir primeiro entretenimento e iluminação decorativa.
ALERTA: consumo supera a geração em 30.0 kWh.
A reserva pode cobrir o déficit atual.
```

```
Previsão para vento de 11 m/s: 27.5 kWh.
```

7. Como o sistema contribui para a colônia

O sistema ajuda a identificar situações de risco antes que falte energia, destaca a necessidade de reduzir cargas não essenciais e evita que os serviços de suporte à vida sejam tratados como cargas comuns. A comparação entre geração, consumo e reserva ajuda a decidir quando economizar ou armazenar energia. A previsão eólica acrescenta uma estimativa do que poderá ser gerado a partir do vento observado.

8. Tecnologias e organização do código

O projeto utiliza Python padrão e não exige instalação de pacotes adicionais. Os arquivos são: main.py (execução integrada), dados.py (dados e hierarquia), decisoes.py (regras e balanço) e previsao.py (regressão linear). O README.md contém instruções para executar e adaptar o sistema.

9. Repositório GitHub Link do repositório público:

https://github.com/stela92s/sistema-colonia-inteligente

10. Conclusão

O projeto integra organização de dados, estruturas hierárquicas, lógica condicional, análise de balanço energético e regressão linear simples. Dessa forma, demonstra a passagem de um sistema que apenas reage aos valores atuais para um sistema que também produz uma estimativa de geração futura. Os resultados dependem da qualidade dos dados e dos limites escolhidos, que podem ser ajustados.